



DeepAMD: Detection and identification of Android malware using high-efficient Deep Artificial Neural Network

Syed Ibrahim Imtiaz^a, Saif ur Rehman^b, Abdul Rehman Javed^{c,*}, Zunera Jalil^c, Xuan Liu^d, Waleed S. Alnumay^e

^a Institute of Avionics and Aeronautics, PAF Complex, E-9, Air University, Islamabad, Pakistan

^b Faculty of Computing and AI, PAF Complex, E-9, Air University, Islamabad, Pakistan

^c Department of CyberSecurity, PAF Complex, E-9, Air University, Islamabad, Pakistan

^d School of Information Engineering, Yangzhou University, China

^e Computer Science Department, King Saud University, Riyadh, Saudi Arabia



ARTICLE INFO

Article history:

Received 20 April 2020

Received in revised form 28 September 2020

Accepted 11 October 2020

Available online 19 October 2020

Keywords:

Android malware
Malware family
Malware category
API calls
Deep learning
Machine learning
Cyberattack
Security

ABSTRACT

Android smartphones are being utilized by a vast majority of users for everyday planning, data exchanges, correspondences, social interaction, business execution, bank transactions, and almost in each walk of everyday lives. With the expansion of human reliance on smartphone technology, cyberattacks against these devices have surged exponentially. Smartphone applications use permissions to utilize various functionalities of the smartphone that can be maneuvered to launch an attack or inject malware by hackers. Existing studies present various approaches to detect Android malware but lack early detection and identification. Accordingly, there is a dire need to craft an efficient mechanism for malicious applications' detection before they exploit the data. In this paper, a novel approach *DeepAMD* to defend against real-world Android malware using deep Artificial Neural Network (ANN) has been adopted including an efficiency comparison of *DeepAMD* with conventional machine learning classifiers and state-of-the-art studies based on performance measures such as accuracy, recall, f-score, and precision. As per the experimental analysis, *DeepAMD* outperforms other approaches in detecting and identifying malware attacks on both Static as well as Dynamic layers. On the Static layer, *DeepAMD* achieves the highest accuracy of 93.4% for malware classification, 92.5% for malware category classification, and 90% for malware family classification. On the Dynamic layer, *DeepAMD* achieves the highest accuracy of 80.3% for malware category classification and 59% for malware family classification in comparison with the state-of-the-art techniques.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

Commercialization of computing services, software, as well as hardware and the emergence of IoT in the last decade has made it possible to access billions of smart devices within no time. The world computing industry has seen a massive increase in the number of personal computers, laptops, smartphones, tablets, and smartwatches during the last four years [1–3]. According to a recent survey [4], a typical user buys a new smartphone every 18–20 months. According to the Groupe Speciale Mobile Association (GSMA) [5] mobile economy report 2020, there are currently 5.2 billion mobile users and 12 billion IoT devices being used. These devices are creating enormous bytes of data and most of this

data is inaccessible due to privacy protections and bandwidth limitations.

Android smartphones are widely used because of its flexible operating system with many user-friendly features. One of the reasons for Android to be one of the most popular operating systems is that it is being supported by Google as open-source software. Android operating system (OS) can run applications developed in Java language. These codes work only if the target OS is Android [6,7]. Android application is a compressed file that comprises of different libraries and records. The Android Manifest file contains the meta-information about the application (i.e., name of the bundle, permissions required, meanings of segments for example services, activities, content providers, broadcast receivers, libraries, and rendition support). Android OS contains a directory named “res” that stores pictures, symbols, and User Interface (UI) formats. Other library resources contain assets that are non-aggregated [8]. Cyberattacks are also evolving with the advancements in computing technology [9,10]. These

* Corresponding author.

E-mail addresses: syedibrahimimtiaz@gmail.com (S.I. Imtiaz), 181065@students.au.edu.pk (S.u. Rehman), abdulehman.cs@au.edu.pk (A.R. Javed), zunera.jalil@mail.au.edu.pk (Z. Jalil), yusuf@seu.edu.cn (X. Liu), wnumay@ksu.edu.sa (W.S. Alnumay).

attacks encompass the violation of Android device security by novel malware attacks that are hard to detect and is challenging.

Android OS contains a module to authorize the permissions required by the Android applications and grant permissions if no breach in security policy occurs. Android authorizations are ordered into four distinct degrees of assurance which are further discussed in [8,11]. Furthermore, the dataset consists of four distinct malware to be classified as: adware [12], ransomware [13], scareware [14], and SMS malware [15]. Due to the evolving and expanding nature of malware, various detection and prevention approaches are being proposed. Researchers have reported two different approaches to detect malware. First is Static analysis, in which applications are checked without their execution and the second is Dynamic analysis, in which malware behavior is analyzed in an isolated environment after execution [16].

Despite the role of modern technologies in improving quality of life and making the cyber world a better place, the surface of cyber-threats and anticipated cyberattacks are at a new level and evolving at a disturbing rate [8,17]. Besides, new attacks are rising every day having the potential to violate the security of the smartphone. The security policy can be violated in different ways according to the operating system being used by the smartphone. In this paper, we focused on Android OS and the associated emerging attacks that can violate its security. It is a challenging task since millions of smartphones are currently running Android OS, therefore, it is the need of time to provide services to the users along with security and privacy.

Some of the existing studies [18–23] used API calls and permissions as a feature to check if malware exists in the smartphone application. They report that Static analysis is not enough to detect malware from obfuscated applications, therefore, Dynamic analysis is also required. Some studies used deep learning to check if malware exists in the smartphone application [24,25]. But they have certain limitations (i.e., low malware detection and identification rate, Dynamic analysis, low detection and identification rate of family and category detection, and identification of malware). Thus, this paper focuses on addressing all these limitations by providing a highly efficient approach for the detection and identification of new types of malware.

In this paper, we make the following contributions:

1. Propose *DeepAMD*, an effective systematic and functional approach to detect and identify Android malware, malware category, and family on both Static and Dynamic layers.
2. Evaluate the effectiveness of *DeepAMD* using the Deep learning approach and conventional machine learning classifiers.
3. Present a comparative analysis of conventional machine learning techniques: Decision tree (J48), Naive Bayesian (NB), Sequential Minimal Optimization (SMO) and Multi-layer Perceptron (MLP) with Deep Artificial Neural Network (Deep ANN).
4. *DeepAMD* effectively enhances detection performance and achieve promising accuracy on both Static as well as Dynamic layers in comparison with conventional machine learning techniques and state-of-the-art studies.

The rest of the paper is organized as follows. Section 2 briefly covers the technical background and recent advancements on Android malware detection and identification. Section 3 provides an extensive discussion on the selected dataset and preliminaries necessary for comprehending the background of Android malware detection and identification. Section 4 presents an overview of our proposed approach *DeepAMD* for Android malware detection and identification, Android permission, intent, and API call features. The experimental setup and results are articulated in Section 5. Section 6 presents comparative analysis. In Section 7, we discussed results and Section 8 concludes along with directions for future work.

2. Related work

Nowadays, smartphones are being used by a huge majority of people in every field of life. A recent study shows the growth of smartphone users from 3.2 billion in 2019 to 3.5 billion in 2020 [26]. Smartphones are not only used to make calls but also store user information in the form of text, images, videos, documents, etc. Android smartphones are trending and contain confidential data that these are a lucrative target for many cybercriminals who try to breach their security and hack user's data [27]. Malware existence on smart devices is increasing and is a big threat requiring the researcher's attention. In the Android framework, security breach occurs mostly by the installation of third-party applications. Android devices have been analyzed to improve security, including presentation judgments, discovery, and investigation of malware.

Authors in [28] evaluated the accuracy of detecting ransomware with machine learning algorithms from the CICAndMal2017 dataset of 10 ransomware families. CICAndMal2017 dataset contains benign and malware applications [29]. CICAndMal2017 has 4 different categories of malware: Adware, Ransomware, Scareware, and SMS Malware. This dataset also contains more than 80 network traffic features. In [30], the CICAndMal2017 dataset was used by the researchers and they chose one PCAP file for each malware family randomly. Next, they extracted features from PCAP files in two stages. First, network flows were separated using a Java program that applies a flow-level technique. Then 15 features were extracted and then used three supervised machine-learning classifiers Random forest, K-Nearest Neighbors, and Decision Tree were used to classify applications as malware, benign, adware, ransomware, or scareware.

CICIDS2017 dataset was compiled using data captured from 3/07/2017 to 7/07/2017 [31]. The authors in [32] proposed the CICID2017 dataset because it worked on the web, Denial of Service Attack (Dos), Distributed Denial of Service (DDoS), Infiltration, Brute Force Secure Shell (SSH), and File Transfer Protocol (FTP) attacks. CICFlowMeter analyzed network traffic that was generated and extracted eighty network flow features. On the bases of FTP and HyperText Transfer Protocol (HTTPS), the CICIDS2017 dataset extracted the basics of behavior of twenty-five users. The authors in [33] evaluated the performance metrics for various detectors by utilizing the CICAndMal2017 dataset. Authors in [34] mentioned a chance of ransomware in the system's API packages of android mobile and proposed "R-PackDroid" that grouped apps of ransomware with high accuracy. The existing work done can be classified into three categories: (1) Application program interface (API) Call Based Android Malware Detection, (2) Intent-Based Android Malware Detection, and (3) Permission-Based Android Malware Detection.

2.1. API call based android malware detection

The authors in [20] utilized API calls as dataset features to check if android applications are malicious or not. They detected obfuscated applications. This research was immediately trailed by Chan and Song [35], where they introduced a feature list of 19 API calls and permission to recognize malware.

Authors in [22] used the CICAndMal2017 dataset that consists of Static features of permissions, intents, as well as Dynamic features of API calls. They enhanced the family classification performance of malware. Afterward, more ways to merge feature sets were introduced. Droidmat [36] introduced a malware analyzer that used Static permissions, intents, and API calls to classify android applications. They utilized k-means to improve their malware ability and utilized k-Nearest Neighbor check if applications are malicious or benign. A few researchers likewise

utilized API sequences, dependency graphs, and bundles. Then, the DroidSIFT [37] malware classifier dependent on the weighted relevant API dependency graph. DroidSIFT put a security-related weighted API graph in a database for each application separately. It was observed that by an efficient graph search in database grouped applications based on the android packages and each group keeps an index of present critical API. An application was tested and its critical graph was generated. From this graph, a feature set was extracted in which if the vector is not zero then it shows the similarity score, and the test applications graph was compared with the database graph. These results were used for anomaly detection and signature detection and had an accuracy of 93% on Genome [38] dataset.

2.2. Intent based android malware detection

Droidmat [36] introduced a malware analyzer that used Static permissions, intents, and API calls to classify android applications. They utilized K-means to improve their malware ability and utilized k-Nearest Neighbor to check if applications are malicious or benign. Similarly, in [27], malware detection and classification model was proposed and used real-world datasets i.e. CICAndMal2017. This model is used for the feature extraction phase and to extract conversation-level features with the PeerShark tool. This model utilized conversation level traffic features of the network.

In [39], a model of mobile malware detection that uses traffic features to check the efficiency of traffic classifier and uses classification techniques for are time-based, flow-based, and packet-based features to check malware families. For feature extraction of the vector, CICFlowMeter [40] was used as the flow generator and had extracted network traffic flow-level features. In [41], authors proposed that intents are effective for identifying malicious applications because intents can encode malware when they are compared with the permissions feature set. Later, it was found through experimentation that the detection rate with permissions was 83% and with intents, it was 91%, when intent features were combined with other features. However, the detection rate was up to 96% with a merge of intents and permissions.

2.3. Permission based android malware detection

In a permission-based approach, the permission list is checked to confirm the existence of malicious apps. Authors in [42] distinguish between benign and malware appreciation using machine learning and monitored permissions and event features. Aung et al. [42] made a malware detection monitor that can recognize malware and benign application. They utilized K-means, random forest, and decision trees to group the malware by selected feature set. Experiments on two distinctive datasets revealed a 90% average detection rate. Likewise, another author Huang et al. [43] guaranteed that a permission-based approach can be utilized as a fast channel that has a detection rate of above 81% on malicious samples.

The evaluation confirmed that network traffic flow-level features are helpful for binary detection in the above-mentioned scenarios. In [44], the malicious samples were collected from CICAndMal2017 with the alignment of permissions. The sequence alignment principle was used to check similarity based on permissions for normal families and malicious families. The classification threshold is obtained from the similarity score between the DNA of the families and the tested application. In [45], the results are evaluated and with those results DL-Droid which uses 31,125 Android apps, and uses 420 Static and Dynamic features. The performance was compared with deep learning-based frameworks and traditional machine learning classifiers.

Several studies provided an overview of the evolution of different malware detection and identification techniques [18–21]. Some studies used deep learning to check if malware exists in the smartphone application [24,25], but had certain limitations (i.e., low malware detection and identification rate, Dynamic analysis, family and category detection, and identification of malware). These limitations arise through time and they have to be updated for the safety of android phone security. This paper focuses on addressing all these limitations by providing a highly efficient approach for the detection and identification of new types of malware. Table 1 shows existing works and the limitations that we addressed in this research.

3. Dataset and preliminaries

We have used recently published Android Malware Dataset (CICInvesAndMal2019) [46]. It is the second part of the dataset CICAndMal2017 [29], in which benign and malware Android applications are tested on real smart devices. This dataset contains several families of Android malware, permissions, and intents as Static features, API calls and all generated log files as Dynamic features. The dataset also includes captured features like process logs, packages, log states, battery states, etc. Table 2 shows a detailed overview of the datasets adopted from [22] that are locally available with published year information. We provide a comparison of our dataset with the features of other publicly available datasets in Table 3. Malware samples in this dataset are classified into four categories: (1) Adware, (2) Ransomware, (3) Scareware, and (4) SMS Malware.

3.1. Adware

Adware is a type of malicious application affecting user privacy and security. Adware may cause the client damage by taking his information and sending it to a remote server, showing advertisements forcefully through screen seizing or showing advertisements in the notification bar, which is typically held for significant framework occasions. In some cases, an Adware may hack the smartphone speaker [12]. The main purpose of Adware is to make the user view or click the maximum unintentional commercials, banners, and posts [47]. Adware is any product bundle that frequently presents ads to clients' history of their application usage or search history. This includes gathering data, frequently and yet sometimes, using this data for malicious intentions [48]. Adware can be utilized deliberately by an advertisement company, other meddling adware may likewise misuse an advertisement company and subvert income and data from the proprietors of the promotion company. Aggressive adware can make alternate routes on to the home screen, take book-marks, change default internet browser, search engine, internet settings, and pushing pointless notices Plankton is one such kind of aggressive adware. Adware can be designed to take control of the user's android device when it is merged with botnet and repacks itself as a popular application [39]. Aggressive Adware can exploit vulnerabilities and attack installed from third-party sources [49,50].

3.2. Ransomware

Ransomware is a sort of malware that requests a cash amount from the tainted client. On Android, there are two general classes of ransomware which are lock-screen and crypto. In lock-screen, class, the smartphone asset is hindered by a picture that completely covers the screen. Secondly, In the crypto class, the ransomware scrambles the client's significant information. Android ransomware normally fits the general meaning of a trojan horse.

Table 1

Summary of existing malware detection studies using deep learning and machine learning.

Year	Ref.	Method	API calls	Intents	Permission	Limitations
2012	[36]	K-means	Yes	Yes	Yes	Limited to Static permission, intents and API calls
2014	[35]	MLP	Yes	No	Yes	Limited features including permissions and API calls
2016	[24]	DBN	Yes	No	Yes	Does not focus on intents and has low detection rate
2014	[25]	DBN	Yes	No	No	Low detection rate

Table 2

Details regarding currently available android malware datasets [22]. Key: Symbolic – S, Continuous – C, SMO – Spider Monkey, PCA – Principal Component Analysis, States – S, Permission – P, Intent – I, Components – C, Certification – Ce, Source Code – SC, API.Call – APC, Network – N.

Dataset name	Pub.	No. of	No. of	Captured static features						Captured dynamic features					Installed On
	Year	Benign	Malware	S	P	I	C	Ce	SC	APC	N	Sys.Call	Infoflow	Log	
Genome [27]	2012	–	1260	×	✓	×	✓	×	✓	×	×	×	×	×	–
Drebin [3]	2014	123,453	5560	×	✓	✓	✓	×	✓	×	×	×	×	×	–
AndroTracker [13]	2015	51,179	4554	×	✓	✓	×	×	✓	×	×	×	×	×	–
SAPIMMDS [11]	2016	1776	906	×	×	×	×	×	×	✓	×	×	×	×	Emulator
Andro-Dumpsys [24]	2016	1776	906	×	×	✓	×	✓	✓	✓	×	×	×	×	Emulator
Andro-Profiler [12]	2016	8840	643	×	×	×	×	×	×	×	×	✓	×	✓	Emulator
Kharon [6]	2016	–	7	×	×	×	×	×	×	×	×	×	✓	×	RealPhone
AAGM [15]	2017	1500	400	×	×	×	×	×	×	×	✓	×	×	×	RealPhone
AMD [23]	2017	–	405	×	×	×	✓	×	✓	×	×	×	×	×	–
MalDozer [14]	2018	38,000	33,000	×	×	×	×	×	✓	×	×	×	×	×	–
UCL [20]	2018	1.2M	–	×	×	×	✓	×	✓	×	×	×	×	×	–
CICAndMal2017	2018	1700	426	✓	✓	✓	×	×	×	✓	✓	×	×	×	RealPhone
CICInvesAndMal2019	2019	5,065	426	✓	✓	✓	×	×	×	✓	✓	×	×	×	RealPhone

Table 3

Comparison of publicly available android malware datasets [22].

Year	Dataset	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	CA12	A13	A14	A15
2012	Genome [27]	S	–	–	✓	–	–	✓	✓	–	✓	✓	×	✓	✓	×
2014	Drebin [3]	S	–	–	✓	–	–	✓	✓	×	✓	✓	✓	✓	×	×
2015	AndroTracker [13]	S	–	–	✓	–	–	×	✓	×	✓	✓	✓	✓	×	×
2016	SAPIMMDS [11]	B	×	✓	✓	×	×	✓	✓	✓	✓	×	×	✓	×	×
2016	Andro-Dumpsys [24]	B	×	✓	✓	×	×	✓	✓	✓	✓	×	✓	✓	×	×
2016	Andro-Profiler [12]	B	×	✓	✓	×	×	✓	×	✓	✓	×	×	✓	×	×
2016	Kharon [6]	B	✓	×	✓	✓	×	✓	×	–	✓	×	×	✓	×	✓
2017	AAGM [15]	D	✓	✓	✓	×	×	✓	×	✓	✓	✓	×	✓	×	✓
2017	AMD [23]	S	–	–	✓	–	–	✓	✓	–	✓	✓	✓	✓	✓	✓
2018	MalDozer [14]	S	×	×	✓	×	×	✓	✓	✓	×	✓	×	✓	×	✓
2018	UCL [20]	S	×	×	✓	×	×	✓	✓	×	×	✓	×	×	×	✓
2017	CICAndMal2017	B	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	×
2019	InvesCICAndMal2019	B	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Legend:

A1: Type of data capturing, Static(S) or Dynamic(D) or both(B).

A2: Utilizing Real-Phone devices instead of emulators.

A3: Having network architecture for the experiment set up.

A4: Examining real-world malware samples.

A5: Having malware activation scenario.

A6: Defining multiple states of data capturing.

A7: Having trust-able fully-labeled malware samples.

A8: Including diverse malware categories and families.

A9: Keeping balance between malicious and benign samples.

A10: Avoiding anonymity and preserving all captured data.

A11: Containing a heterogeneous set of resources.

A12: Providing a variety of feature sets for other researchers.

A13: For meta-data, includes a proper documentation.

A14: Including malware taxonomy.

A15: Being up-to-date.

At times, the vindictive APKs duplicate just the name and symbol of the typical application or mask it as an authentic document in an SMS or email. Social engineering is to control exploited people from introducing malicious APKs and executing functions [13]. Ransomware is a file that can encrypt the files and lock the device, then demand payment from the end-users for decryption of files or to unlock the device [51]. When the files are encrypted once, these cannot be recovered with the decryption key even if the ransomware is removed from the system [52]. Ransomware works with permissions and intents on the android system [21]. Ransomware can also use identity or label for an authentic crime investigation organization (USA Crime investigation or FBI) and make fake claims to mislead target users [53].

3.3. Scareware

Scareware is the type of malware that aims to scare the end-user paying for useless applications [54]. Scareware is made to scare or to trap users with some phishing website to steal their information [55]. Scareware trick the user by presenting scam as legitimate applications that typically take on the appearance of security applications for example, “against malware programming” or more explicitly “hostile to infection programming”. Scareware is particularly made to incorporate phony filtering exchanges, fake advancement bars, and phony alarms. Scareware may show counterfeit arrangements of virus records That are so produced incorporate documents that may not exist on the PC or

Table 4

Families with their count samples with respect to their designated category. Key: Captured Samples – CS.

Adware		Ransomware		Scareware		SMS Malware	
Family	CS	Family	CS	Family	CS	Family	CS
Dowgin	10	Charger	10	AndroidDefender	17	BeanBot	9
Ewind	10	Jisut	10	AndroidSpy.277	6	Biige	11
Feiwo	15	Koler	10	AV for Android	10	FakeInst	10
Gooligan	14	LockerPin	10	AVpass	10	FakeMart	10
Kemoge	11	Simplocker	10	FakeApp	10	FakeNotify	10
koodous	10	Pletor	10	FakeApp.AL	11	Jifake	10
Mobidash	10	PornDroid	10	FakeAV	10	Mazarbot	9
Selfmite	4	RansomBO	10	FakeJobOffer	9	Nandrobox	11
Shuanet	10	Svpeng	11	FakeTaoBao	9	Plankton	10
Youmi	10	WannaLocker	10	Penetho	10	SMSsniffer	9
–	–	–	–	VirusShield	10	Zsone	10

they might be inconsistent with the working framework. Attackers can steal the credentials of organizations and individuals to cause harm or breach security [14].

3.4. SMS malware

SMS malware takes control of messages on an Android device and sends unwanted messages. This malware is being used by the owner to send messages by the infected mobile phones on his command [56]. SMS attacks include the making and spreading of malware by attackers, intended to focus on a hacked individual's smartphone. The trojan can be intended to make unapproved calls or send unapproved messages without the client's information or assent. These calls and messages are consequently coordinated to chargeable SMS content administrations or premium-charge numbers used by the cyber-criminal for creating huge income streams for cyber-criminal systems [57] (see Table 4).

4. Android malware detection and identification

In this section, we present our proposed approach named *DeepAMD* for malicious application detection. *DeepAMD* comprises feature extraction, classification of application as malicious or benign, classification of malware category, and malware family. Fig. 1 summarizes our proposed approach which consists of Static binary classification and Dynamic malware classification. First, in the Static layer, the samples are classified as malware and benign. Then in the Dynamic layer, the samples that are classified by the Static layer as malware are further classified into different 4 categories (Adware, Ransomware, SMS Malware, and Scareware) and 39 families.

4.1. Pre-processing

Pre-processing is a preliminary step to obtain the best performance in any machine learning model. Pre-processing steps typically include removal of NaN, duplicate instances, and normalization/scaling. The selected dataset has low variance and ambiguities therefore, we choose MinMax scaling for feature normalization. Normalizing refers to the re-scaling of real-valued numeric attributes to a fixed range (e.g., 0 and 1). It is highly important to scale the input attributes for a model that relies on the magnitude of values. MixMax scaling normalizes data using the formula mentioned in (1).

$$X_{norm} = \frac{X_i - X_{min}}{X_{max} - X_{min}} \quad (1)$$

where X_i is the original value of the feature that is subtracted from minimum value of that feature and divided by subtracted result of maximum and minimum of the feature.

4.2. Feature extraction

CICInvesAndMal2019 [46] is a two-layered dataset consisting of Static and Dynamic layers for malware detection. The first layer is the Static layer that analyzes malware applications and if it detects any malicious application then that application is also considered malware for the Dynamic layer. If the Static-base first layer detects a suspicious malware, there is more possibility of malicious intentions in that sample. We believe if a sample is detected suspicious with a Static layer, the analyzer should consider it as malware for the next layer as well. As a result, we can reduce the risk of trusting unknown samples. In the dataset, there are testing as well as training samples of both Static and Dynamic layers.

4.3. Machine learning models

In this study, we use the following machine learning algorithms to evaluate and compare the effectiveness of our proposed *DeepAMD* approach:

Naïve Bayes (NB): This is the most straightforward and fast classification algorithm suitable for a large chunk of data. It uses Bayes theorem of probability for prediction of unknown class.

Sequential Minimal Optimization (SMO): This algorithm works by separating hyperplane and the input to SMO is represented as a set of points in the space that are mapped such that each point belonging to a class is separated efficiently by analyzing gaps. SMO comprises of kernel function that provides information a gap between points.

Multilayer Perceptron (MLP): This is a class of Artificial Neural Networks that uses backpropagation for training and its multiple layers and non-linear activation distinguishes it from a linear perceptron algorithm [9].

Decision tree (DT/J48): This supervised learning algorithm continuously split data according to a certain parameter. The tree can be explained by two entities, namely decision nodes and leaves. The leaves are the decisions or outcomes, and the decision nodes are points where the data is split.

Deep Artificial Neural Networks (Deep ANN): Artificial Neural Networks (ANN) are multi-layer fully-connected neural nets and consist of an input layer, multiple hidden layers, and an output layer. Every node in one layer is connected to every other node in the next layer. We make the network deeper by increasing the number of hidden layers. Deep ANNs are used for many real-world applications nowadays due to their efficient performance. The success of deep learning during the last decade is due to a combination of improved theory starting with unsupervised pre-training and deep belief nets and improved hardware resources such as general-purpose graphics processing units (GPUs). Deep ANNs are now routinely used with impressive results in areas such as image analysis, pattern recognition, object detection, natural language processing, and self-driving cars to name a few areas.

4.4. Hyper-parameter tuning for android malware detection and identification using Deep ANN

We use Deep ANN such that the 8114-axis data that corresponds to the input layer of 8114 dimensions correspond to the 2, 4, and 42 axes of the output layer for the malware binary classification of malware and benign class labels, category classification of “adware”, “ransomware”, “scareware” and “SMS Malware” and family classification of families i.e. “dowgin”, “ewind”, “feiwo”, and others respectively for the detection and identification of attack over Static layer network traffic. For attack detection and identification over network traffic on the Dynamic layer, the

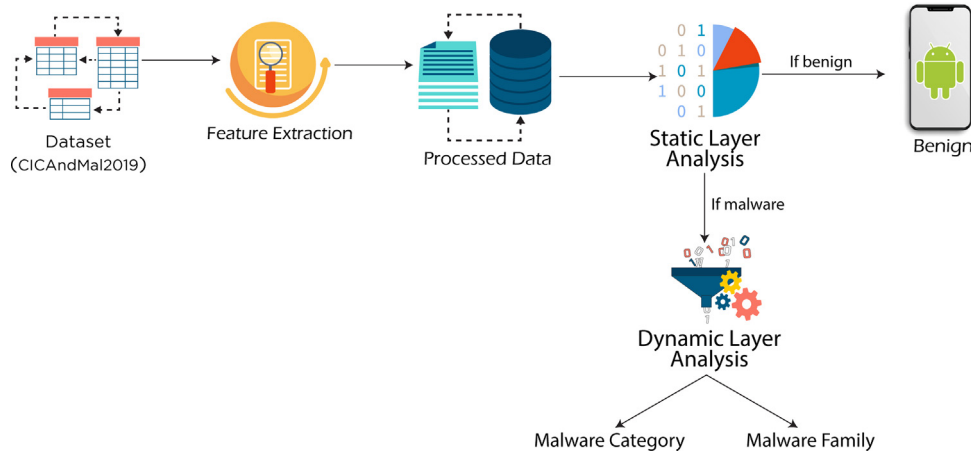


Fig. 1. Graphical representation of proposed android malware application detection and identification approach.

Table 5

Details of the optimal hyper-parameters deep artificial neural network.

Parameter	Value
Initial Bias	0
Internal layer	3
Dropout	Dynamic
Activation function at all layers	Relu
Activation function at output layer	Sigmoid
Batch size	Dynamic
Learning optimizer	Adam
Error function	Binary & Categorical Cross Entropy

Table 6

Computing environment.

Parameter	Value
Operating System	Windows 10 Professional 1909
CPU	Intel(R) Core(TM)i7-6700HQ
RAM	16 GB
GPU	NVIDIA GeForce 1060
CUDA Version	9.0
Python Version	3.8

919-axis data correspond to the input layer of 919 dimensions correspond to the 4 and 40 axes of the output layer for the category classification of “adware”, “ransomware”, “scareware” and “SMS malware” and family classification of families i.e., “dowgin”, “ewind”, “feiwo” and others respectively. Neurons on the internal layer are computed using the “Relu” unit. The output layer is defined by a “sigmoid” function, error, and cross-entropy function. Details of this Deep ANN are summarized in Table 5

4.5. Comparative methods

Machine learning algorithms are used for the detection and identification of Android malware. These algorithms include MLP, NB, J48, SMO. These algorithms are used to compare the results with our proposed *DeepAMD* technique. The SMO is used with various parameters and finally set to the PUK kernel and the value of gamma and alpha is set to 1.0. NB is used with a kernel estimator. J48 is used with a confidence factor of 2.5. Last, we analyze the performance of these algorithms using evaluation measures explained in Section 5.1.

5. Evaluation and results

To evaluate the performance of the proposed *DeepAMD*, we perform two major tasks: (i) detection and identification, aims at checking if a given app is a malware or not, (ii) attribution, which aims at determining the family of detected malware. CICAndMal2019 [46] has multiple levels of classification. At the first level, the dataset is classified into malware and benign samples. In the second level, the dataset is classified into 4 categories of malware that are adware, ransomware, scareware, SMS malware, and benign samples. The dataset is further labeled into 42 Families of these 4 malware categories. The previous version of the dataset CICAndMal2017 [29] initially had 39 categories.

5.1. Evaluation measures and experimental setup

Evaluation measure and experimental setup play a vital role to analyze the performance of a machine learning model. For the experiment, we split data and 80% of data is used for training, and the rest 20% for testing. We computed accuracy, precision, recall, and f-score to evaluate the performance of the proposed method. Details can be seen in the Equations given below. Table 6 presents the computing environment used for experiments.

$$\text{accuracy} = \frac{TP_{\text{Anomaly}} + TN_{\text{Normal}}}{TP_{\text{Anomaly}} + FN_{\text{Anomaly}} + TN_{\text{Normal}} + FP_{\text{Normal}}} \quad (2)$$

$$\text{precision} = \frac{TP_{\text{Anomaly}}}{TP_{\text{Anomaly}} + FP_{\text{Anomaly}}} \quad (3)$$

$$\text{recall} = \frac{TP_{\text{Anomaly}}}{TP_{\text{Anomaly}} + FN_{\text{Normal}}} \quad (4)$$

$$f - \text{score} = 2 \times \frac{\frac{TP_{\text{Anomaly}}}{TP_{\text{Anomaly}} + FN_{\text{Normal}}} \times \frac{TP_{\text{Anomaly}}}{TP_{\text{Anomaly}} + FP_{\text{Anomaly}}}}{\frac{TP_{\text{Anomaly}}}{TP_{\text{Anomaly}} + FN_{\text{Normal}}} + \frac{TP_{\text{Anomaly}}}{TP_{\text{Anomaly}} + FP_{\text{Anomaly}}}} \quad (5)$$

Here:

- True Positives (TP): Predicted case to be in YES and is actually in it.
- False Positives (FP): Predicted case to be in YES but is not actually in it.
- True Negative (TN): Predicted case not to be in YES and is not actually in it.
- False Negative (FN): Predicted case not to be in YES, but is actually in it.

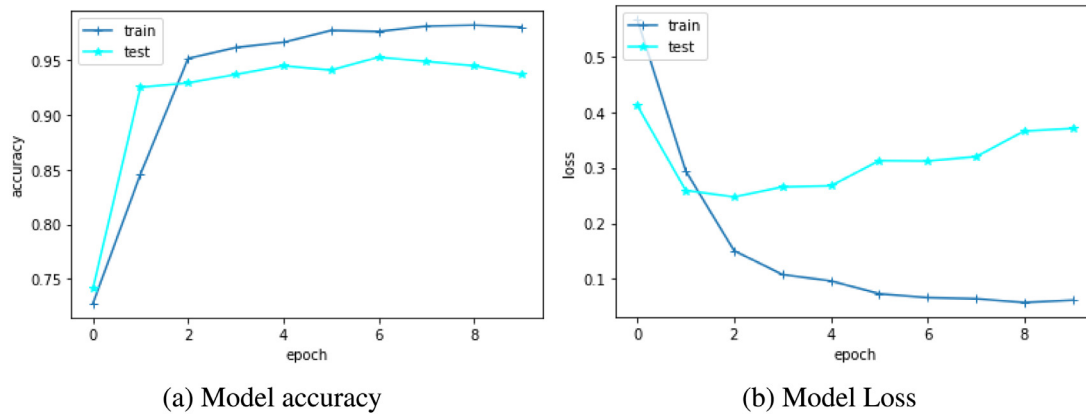


Fig. 2. Model accuracy and loss of binary classification on Static layer using train and validation datasets.

Table 7

Binary malware classification performance on static layer.

	Accuracy (%)	F-score (%)	Recall (%)	Precision (%)
J48	0.905	0.906	0.905	0.906
NB	0.620	0.634	0.620	0.809
SMO	0.918	0.913	0.918	0.926
MLP	0.905	0.906	0.905	0.906
DeepAMD	0.934	0.932	0.934	0.935

5.2. Malware binary detection on Static layer

Table 7 demonstrates the result on Static Malware Binary dataset. *DeepAMD* approach gets the best accuracy of 93.41% using *DeepAMD*. Other conventional techniques: J48, NB, SMO and MLP achieves the accuracy of 90.5%, 62.0%, 91.8% and 90.5% respectively. NB obtained the least accuracy because NB needs more data instances to work better as it works on probability distribution. J48, SMO and MLP perform quite well on Static binary dataset. The Cohens Kappa of *DeepAMD* is reported as 82.7%. *DeepAMD* show a significant gain of performance in comparison with existing methodologies. *DeepAMD* shows a proficient gain of 0.3% and 0.6% in terms of accuracy and f-score respectively.

Fig. 2a depicts the accuracy convergence of *DeepAMD* concerning epochs. *DeepAMD* achieves the highest test accuracy of 0.934% at the 6th epoch. Training accuracy starts at 0.7% and goes up to 0.985%. Then the convergence of training accuracy becomes stable. Test accuracy starts at 0.75% and goes up to 0.934%. It slightly went down at the 4th and 6th epoch. Fig. 2b depicts the convergence of loss of *DeepAMD* concerning epochs. *DeepAMD* achieves the lowest loss of 0.25% at the 2nd epoch. Training loss starts at 0.6 and goes down to 0.05%. Then the convergence of training loss becomes stable. Test loss starts at 0.41 and goes up to 0.33%. Then the convergence of test loss becomes irregular against the test data as epochs increase.

Fig. 3 depicts the confusion matrix of the *DeepAMD*. It shows how many instances of normal applications are getting confused with instances of malicious application. Only 0.0697% of the normal instances are getting confused with malicious instances while on the other hand only 0.0533% of the malicious instances are getting confused with normal instances.

5.3. Malware category detection on Static layer

Table 8 demonstrates the result of the Static Malware Category dataset. *DeepAMD* achieves the best accuracy of 0.925% using *DeepAMD*. Other conventional techniques: J48, NB, SMO, and MLP achieve the accuracy of 0.893%, 0.561%, 0.868%, and 0.724% respectively. NB achieves the minimum accuracy of 0.561% because

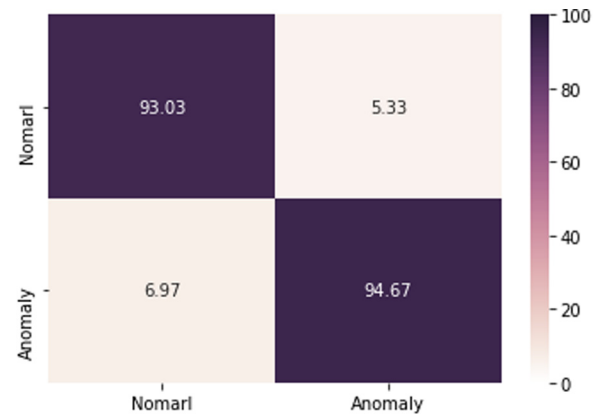


Fig. 3. Confusion matrix of binary classification on Static layer.

Table 8

Malware category classification performance on static layer.

	Accuracy (%)	F-score (%)	Recall (%)	Precision (%)
J48	0.893	0.893	0.893	0.893
NB	0.561	0.61	0.561	0.561
SMO	0.868	0.868	0.868	0.868
MLP	0.724	0.724	0.724	0.724
DeepAMD	0.925	0.921	0.925	0.922

NB needs more data instances as it works on the probability distribution. The Cohens Kappa is calculated to be 0.824% and the ROC AUC curve is 0.941% in this analysis for Category as a feature in malware dataset. *DeepAMD* shows a significant gain of performance in comparison with existing methodologies. *DeepAMD* shows a proficient gain of 0.038% in terms of f-score.

Fig. 4a depicts the accuracy convergence of *DeepAMD* with respect to epochs. *DeepAMD* achieve the highest accuracy of 0.925% at 14th epoch. Training accuracy starts at 0.75% and goes up to 0.97% and then becomes stable. Test accuracy starts at 0.75% and goes up to 0.925%. It slightly went down at the 8th epoch. Fig. 4b depicts the convergence of the accuracy of *DeepAMD* concerning epochs. *DeepAMD* achieves the lowest loss of 0.1% at 17th epoch. Training loss starts at 0.9% and goes down to 0.05%. Then the convergence of training loss becomes stable. Test loss starts at 0.72% and goes down to 0.45%. Fig. 5 depicts the confusion matrix of the category classification on static layer.

5.4. Malware family detection and identification on Static layer

Table 9 demonstrates the result of the Static Malware Family feature set. *DeepAMD* achieves the best accuracy of 0.90% in the

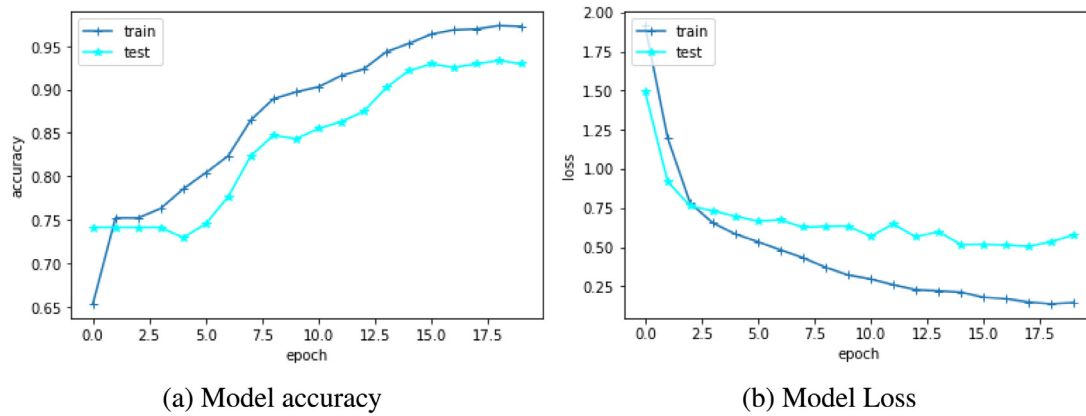


Fig. 4. Model accuracy and loss of category classification using train and validation datasets on Static layer.



Fig. 5. Confusion matrix of category classification on Static layer.

Table 9

Malware family classification performance on static layer.

	Accuracy (%)	F-score (%)	Recall (%)	Precision (%)
J48	0.862	0.863	0.862	0.884
NB	0.695	0.753	0.695	0.855
SMO	0.836	0.780	0.836	0.756
MLP	0.692	0.685	0.692	0.683
DeepAMD	0.900	0.896	0.899	0.904

Static layer for family malware detection. Other conventional techniques: j48, NB, SMO and MLP achieves the accuracy of 0.862%, 0.695%, 0.836%, and 0.692% respectively. In this case, the Algorithm Naive Bayes achieves the minimum accuracy of 0.695% because Naive Bayes needs more data instances as it works on the probability distribution. The Cohens Kappa is calculated to be 0.783% and the ROC AUC curve is 0.918% in this analysis for Family as a feature in the malware dataset.

Fig. 6a depicts the convergence of the accuracy of *DeepAMD* concerning epochs. *DeepAMD* achieves the highest accuracy of 0.90% at the 14th epoch. Training accuracy starts at 0.75% and goes up to 0.95%. The convergence of training accuracy increases after each epoch. Test accuracy starts at 0.75% and goes up to 0.95%. It slightly went down at the 8th epoch. Fig. 6b depicts the convergence of the accuracy of *DeepAMD* concerning epochs. *DeepAMD* achieves the lowest loss of 0.80% at the 14th epoch. Training loss starts at 0.016% and goes down to 0.2%. Then the convergence of training loss becomes stable. Test loss starts at 0.013% and goes down to 0.8%.

Table 10

Malware category classification performance on dynamic layer.

	Accuracy (%)	F-score (%)	Recall (%)	Precision (%)
J48	0.712	0.713	0.712	0.720
NB	0.727	0.723	0.727	0.731
SMO	0.681	0.701	0.681	0.781
MLP	0.575	0.538	0.575	0.512
DeepAMD	0.803	0.805	0.803	0.822

5.5. Malware category detection and identification on Dynamic layer

Table 10 shows the result of the Dynamic Malware Category feature set. *DeepAMD* achieves the best accuracy of 0.803% using *DeepAMD*. Other conventional techniques: J48, NB, SMO, and MLP achieve the accuracy of 0.712%, 0.727%, 0.681%, and 0.575% respectively. *DeepAMD* shows a significant gain of performance in comparison with existing methodologies. *DeepAMD* shows a proficient gain of 0.076%, 0.092%, 0.076% and 0.041% in terms of accuracy, f-score, recall, and precision respectively. The Cohens Kappa is calculated to be 0.727% and the ROC AUC curve is 0.823% in this analysis for the family as a feature in the malware dataset.

Fig. 7a depicts the convergence of the accuracy of *DeepAMD* concerning epochs. *DeepAMD* achieves the highest accuracy of 0.803% at the 12th epoch. Training accuracy starts at 0.1% and goes up to 0.90%. Then the convergence of training accuracy becomes stable. Test accuracy starts at 0.33% and goes up to 0.82%. It slightly went down at the 4th epoch. Fig. 7b depicts the convergence of the accuracy of *DeepAMD* concerning epochs. *DeepAMD* achieves the lowest loss of 0.6% at the 8th epoch. Training loss starts at 0.014% and goes down to 0.2%. Then the convergence of training loss becomes stable. Test loss starts at 0.014% and goes down to 0.62%. Fig. 8 depicts the confusion matrix of the category classification on dynamic layer.

5.6. Malware family detection and identification on Dynamic layer

Table 11, shows the result on Dynamic Malware Family feature set we get the best accuracy of 0.59% using *DeepAMD*. Other conventional techniques: J48, SMO, MLP and NB achieves the accuracy of 0.459%, 0.262%, 0.49% and 0.524% respectively. *DeepAMD* show a significant gain of performance in comparison with existing methodologies. *DeepAMD* shows a proficient gain of 0.07%, 0.058%, and 0.066% in terms of accuracy, f-score, and recall respectively. The Cohens Kappa is calculated to be 0.507% and the ROC AUC curve is 0.558% in this analysis for family as feature in malware dataset.

Fig. 9a depicts the convergence of the accuracy of *DeepAMD* concerning epochs. *DeepAMD* achieves the highest accuracy of

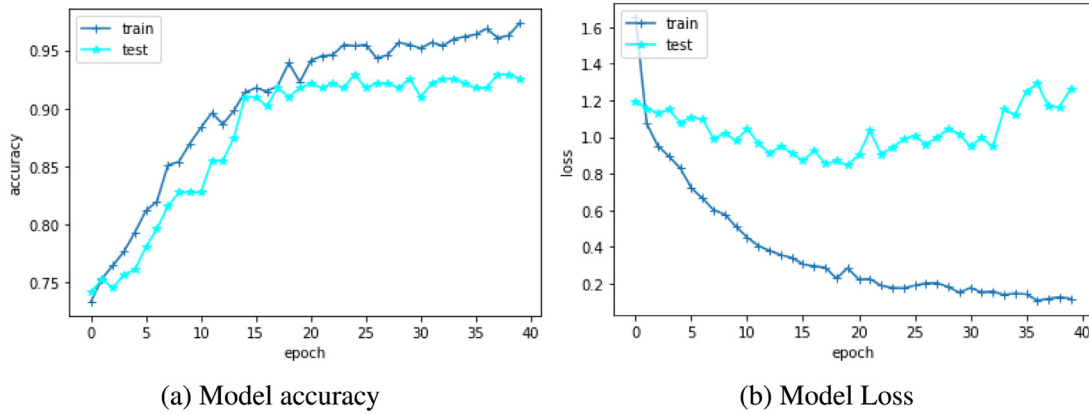


Fig. 6. Model accuracy and loss on train and validation datasets of family classification on Static layer.

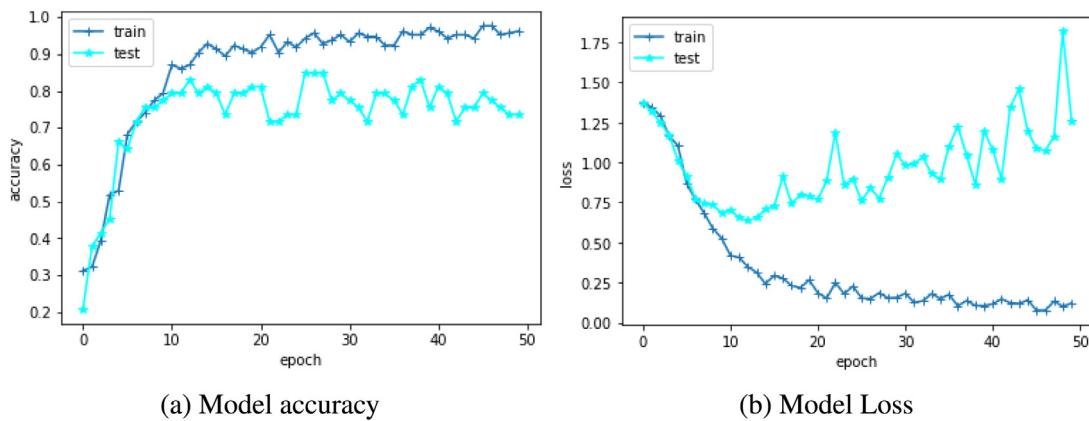


Fig. 7. Model accuracy and loss of category classification using train and validation datasets on Dynamic layer.

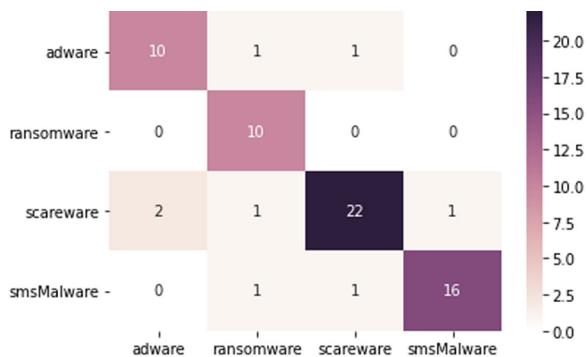


Fig. 8. Confusion matrix of category classification on Dynamic layer.

Table 11
Malware family classification performance of *DeepAMD* on dynamic layer.

	Accuracy (%)	F-score (%)	Recall (%)	Precision (%)
J48	0.442	0.479	0.442	0.603
NB	0.590	0.581	0.590	0.650
SMO	0.262	0.259	0.262	0.334
MLP	0.049	0.040	0.049	0.076
<i>DeepAMD</i>	0.557	0.540	0.55	0.591

0.590% at the 44th epoch. Training accuracy starts at 0.05% and goes up to 0.9%. Then the convergence of training accuracy becomes stable. Test accuracy starts at 0.1% and goes up to

0.64%. It slightly went down at the 40th epoch. Fig. 9b depicts the convergence of the accuracy of *DeepAMD* concerning epochs. *DeepAMD* achieves the lowest loss of 0.75% at the 9th epoch. Training loss starts at 0.016% and goes down to 0.2%. Then the convergence of training loss becomes stable. Test loss starts at 0.013% and goes down to 0.8%.

6. Comparative analysis

In Tables 12, 13, 14, precision and recall are compared with 2 different versions of a dataset. Case A represents the dataset CICAndMal2017 [29]. Case B represents the second version of CICAndMal2017 [46]. The research was conducted by author [22], in which they used random forest algorithm to calculate the precision and recall of dataset. Our approach *DeepAMD* achieves the highest accuracy using the *DeepAMD* algorithm. As compared to other studies [22,27,32], we improve our results in both the Static and Dynamic layers with our analysis. We achieve the highest precision 93.5% as shown in Table 12 for malware binary classification, slightly low accuracy from state-of-the-art for malware category classification, and best accuracy of 65.1% using our approach as shown in Table 14. on the Static layer, we improve the performance of binary classification of binary malware by 3.0% and on the Dynamic layer, we improve the performance of classification of malware family by 6.1% within comparison with state-of-the-art.

In Table 12, *DeepAMD* achieve the highest precision of 98.3% using SMO for Static layer malware binary classification. Other

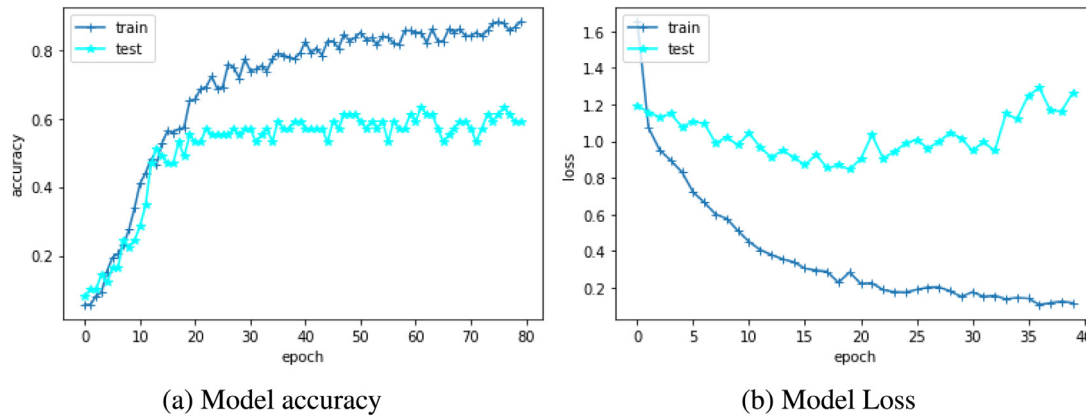


Fig. 9. Model accuracy and loss of family classification using train and validation datasets on Dynamic layer.

Table 12

Result comparison of malware binary classification on static layer.

Dataset	Precision	Recall
A [22]	85.8% (RF)	88.3% (RF)
A [27]	89% (RF)	83.22% (RF)
A [58]	85.4% (KNN)	88.1% (KNN)
A [58]	85.1% (DT)	88% (DT)
A [27]	85.7% (DT)	86.1% (DT)
B [22]	95.3% (RF)	95.3% (RF)
B DeepAMD	93.5%	93.4%

Table 13

Result comparison of malware category classification on dynamic layer.

Dataset	Precision	Recall
A [22]	49.9% (RF)	48.5% (RF)
A [27]	80.2% (RF)	79.6% (RF)
A [58]	49.5% (KNN)	48% (KNN)
A [58]	47.8% (DT)	45.9% (DN)
A [27]	77% (DT)	77% (DT)
B [22]	83.3% (RF)	81% (RF)
B DeepAMD	82.2%	80.3%

Table 14

Result comparison of malware family classification on dynamic layer.

Dataset	Precision	Recall
A [22]	27.5% (RF)	27.5% (RF)
A [58]	26.66% (DT)	20.06% (DT)
A [58]	27.24% (KNN)	23.74% (KNN)
B [22]	59.7% (RF)	61.2% (RF)
B DeepAMD	65% (NB)	59% (NB)

studies [22,27,58] achieve the highest precision of 95.3%, 86.6%, 85.4% respectively. In Table 13, DeepAMD achieve the highest precision of 82.2% using Deep_ANN for Dynamic layer malware category classification. Other studies [22,27,58] achieve the highest precision of 83.3%, 80.2%, 49.5% respectively. In Table 14, DeepAMD achieve the highest precision of 65.1% using Deep_ANN for Dynamic layer malware family classification. Other studies [22, 58] achieve the highest precision of 59.7% and 27.24% respectively.

7. Discussion

Existing studies present various approaches to analyzing the permission structures but have certain limitations. We performed both Static and Dynamic analysis on the features to determine the efficiency of the dataset. For malicious application detection and identification, DeepAMD applies a series of steps: feature extraction, data balancing, classification of application into the

malicious or benign application, classification of malware category, and classification of a malware family. In pre-processing, we removed NaN values, duplicate instances, and applying normalization. We observed that our dataset has low variance and ambiguities therefore, we choose MinMax scaling for feature normalization. We used CICInvesAndMal2019 is a two-layered framework. The first layer is the Static layer that analyzes malware applications and if it detects any malicious then that application is also considered malware for a Dynamic layer. If the Static-base first layer detects a suspicious malware, there is more possibility of malicious intentions in that sample. We believe that if a sample is detected suspicious with a Static layer, the analyzer should consider it as malware for the next layer. As a result, we can reduce the risk of trusting unknown samples. In the dataset, there are testing and training samples of Static and Dynamic layers.

The results in Table 7 demonstrate that the accuracy and F1-score for classification of malware from benign are highest using DeepAMD on the Static layer. The DeepAMD can classify malware from benign with 93.4% accuracy and 87.8% F1-score. When we applied DeepAMD on malware category features on the Static layer comprising of different categories of malware which include adware, ransomware, scareware, SMS malware, premium SMS, and benign. Table 8 deduce that DeepAMD is the ideal algorithm for classification and it worked with the accuracy of 93.1%. In Table 9 the accuracy obtained using the DeepAMD algorithm is 93.4% when DeepAMD classifies different malware families on the Static layer. On the Dynamic layer, in Table 10, the DeepAMD performed well for classification with an accuracy of 93.4% while Table 11 shows the best accuracy of malware family classification. DeepAMD outperforms all other algorithms and achieves the highest accuracy of 59%. The results of the proposed research concluded that with our approach DeepAMD, the highest accuracy achieved is with the DeepAMD. As compared to other studies [22,32], DeepAMD improves the results on both Static and Dynamic layers. On the Static layer, DeepAMD improves the precision of binary classification of binary malware to 98.3% and on the Dynamic layer, DeepAMD improves the precision of family classification to 65.1% with our research.

F-score is a better measure to analyze the efficiency of machine learning models in case of imbalanced data than the accuracy matrix [59]. The existing studies did not unitize this factor for this dataset. According to the results shown in existing studies, they only considered precision and recall which is an important point to note. In this research, the DeepAMD approach achieves the highest f-score of 93.2% for the Static layer in binary malware classification. Similarly in the Static layer, DeepAMD achieves the f-score for malware category and family classification performance of 92.1% and 89.6% respectively. In the case

of a Dynamic layer, the highest f-score of malware category is 80.5% achieved by *DeepAMD* and for the highest f-score of malware family classification is 58.1% achieved by the Naive Bayes algorithm.

8. Conclusion

The evolution of the Internet of Things (IoT) and smart devices has brought a new range of challenges to device vendors, software developers as well as cybersecurity professionals. Previously unobserved Android malware is being identified and new malware is evolving. To counter malware in Android devices, in this paper, a novel *DeepAMD* approach is proposed. *DeepAMD* achieved the highest accuracy of 93.4% on the Static layer to classify binary malware and then categorize malware using *DeepAMD*. *DeepAMD* achieved an accuracy of 93.1% on the Static layer to classify malware families using the *DeepAMD*. Secondly, on the Dynamic layer, we achieved the highest accuracy of 80.3% for malware category classification. We achieved an accuracy of 59.0% for malware family classification on the Dynamic layer. The *DeepAMD* is evaluated using the state-of-the-art CICAndMal2019 dataset and experimental results demonstrated that *DeepAMD* is the most efficient method for detecting and identifying Android malware on the Static as well as Dynamic layer. In the future, we intend to make an online service through which the user would be able to see if an application is benign or malicious before downloading it. This step would contribute positively to ensure the security of an android smartphone device.

CRediT authorship contribution statement

Syed Ibrahim Imtiaz: Acquisition of data, Writing - original draft. **Saif ur Rehman:** Acquisition of data, Analysis and/or interpretation of data, Writing - original draft. **Abdul Rehman Javed:** Conception and design of study, Acquisition of data, Writing - original draft. **Zunera Jalil:** Conception and design of study, Analysis and/or interpretation of data, Writing - original draft, Writing - review & editing. **Xuan Liu:** Writing - review & editing. **Waleed S. Alnumay:** Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research is supported by Researchers Supporting Project number (RSP-2020/250), King Saud University, Riyadh, Saudi Arabia. All authors approved the version of the manuscript to be published.

References

- [1] Gartner says worldwide sales of smartphones recorded first ever decline during the fourth quarter of 2017, 2020, <https://www.gartner.com/en/newsroom/> (Accessed: 2020-03-12).
- [2] G. Raja, A. Ganapathisubramanian, S. Anbalagan, S.B.M. Baskaran, K. Raja, A.K. Bashir, Intelligent reward-based data offloading in next-generation vehicular networks, *IEEE Internet Things J.* 7 (5) (2020) 3747–3758.
- [3] A. Rehman Javed, Z. Jalil, S. Atif Moqurrah, S. Abbas, X. Liu, Ensemble adaboost classifier for accurate and fast detection of botnet attacks in connected vehicles, *Trans. Emerg. Telecommun. Technol.* (2020) e4088.
- [4] E. Lavoie, L. Hendren, Personal volunteer computing, in: *Proceedings of the 16th ACM International Conference on Computing Frontiers*, 2019, pp. 240–246.
- [5] The mobile economy 2020, 2020, <https://www.gsma.com/mobileeconomy/> (Accessed: 2020-03-12).
- [6] K. Sharma, B. Gupta, Towards privacy risk analysis in android applications using machine learning approaches, *Int. J. E-Serv. Mob. Appl. (IJESMA)* 11 (2) (2019) 1–21.
- [7] A.R. Javed, M.U. Sarwar, S. Khan, C. Iwendi, M. Mittal, N. Kumar, Analyzing the effectiveness and contribution of each axis of tri-axial accelerometer sensor for accurate activity recognition, *Sensors* 20 (8) (2020) 2216.
- [8] C. Wang, Q. Xu, X. Lin, S. Liu, Research on data mining of permissions mode for android malware detection, *Cluster Comput.* 22 (6) (2019) 13337–13350.
- [9] A.R. Javed, M.O. Beg, M. Asim, T. Baker, A.H. Al-Bayatti, Alphalogger: detecting motion-based side-channel attack using smartphone keystrokes, *J. Ambient Intell. Humaniz. Comput.* (2020) 1–14.
- [10] M. Mittal, C. Iwendi, S. Khan, A. Rehman Javed, Analysis of security and energy efficiency for shortest route discovery in low-energy adaptive clustering hierarchy protocol using levenberg-marquardt neural network and gated recurrent unit for intrusion detection system, *Trans. Emerg. Telecommun. Technol.* (2020) e3997.
- [11] N.T. Cam, V.-H. Pham, T. Nguyen, Detecting sensitive data leakage via inter-applications on android using a hybrid analysis technique, *Cluster Comput.* 22 (1) (2019) 1055–1064.
- [12] I. Ideses, A. Neuberger, Adware detection and privacy control in mobile devices, in: *2014 IEEE 28th Convention of Electrical & Electronics Engineers in Israel (IEEEI)*, IEEE, 2014, pp. 1–5.
- [13] J.-S. Ko, J.-S. Jo, D.-H. Kim, S.-K. Choi, J. Kwak, Real time android ransomware detection by analyzed android applications, in: *2019 International Conference on Electronics, Information, and Communication (ICEIC)*, IEEE, 2019, pp. 1–5.
- [14] M. Sikorski, A. Honig, *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*, no starch press, 2012.
- [15] F. Faghihi, M. Abadi, A. Tajoddin, Smsbothunter: A novel anomaly detection technique to detect sms botnets, in: *2018 15th International ISC (Iranian Society of Cryptology) Conference on Information Security and Cryptology (ISCISC)*, IEEE, 2018, pp. 1–6.
- [16] A. Arora, S. Garg, S.K. Peddoju, Malware detection using network traffic analysis in android based mobile devices, in: *2014 Eighth International Conference on Next Generation Mobile Apps, Services and Technologies*, IEEE, 2014, pp. 66–71.
- [17] C. Iwendi, Z. Jalil, A.R. Javed, T. Reddy, R. Kaluri, G. Srivastava, O. Jo, Keysplitwatermark: Zero watermarking algorithm for software protection against cyber-attacks, *IEEE Access* 8 (2020) 72650–72660.
- [18] M. Shafiq, Z. Tian, A.K. Bashir, X. Du, M. Guizani, Iot malicious traffic identification using wrapper-based feature selection mechanisms, *Comput. Secur.* (2020) 101863.
- [19] J. Yu, T. Yamauchi, Access control to prevent attacks exploiting vulnerabilities of webview in android os, in: *2013 IEEE 10th International Conference on High Performance Computing and Communications & 2013 IEEE International Conference on Embedded and Ubiquitous Computing*, IEEE, 2013, pp. 1628–1633.
- [20] Y. Nishimoto, N. Kajiwar, S. Matsumoto, Y. Hori, K. Sakurai, Detection of android api call using logging mechanism within android framework, in: *International Conference on Security and Privacy in Communication Systems*, Springer, 2013, pp. 393–404.
- [21] S. Song, B. Kim, S. Lee, The effective ransomware prevention technique using process monitoring on android platform, *Mob. Inf. Syst.* 2016 (2016).
- [22] L. Taheri, A.F.A. Kadir, A.H. Lashkari, Extensible android malware detection and family classification using network-flows and api-calls, in: *2019 International Carnahan Conference on Security Technology (ICCST)*, IEEE, 2019, pp. 1–8.
- [23] F. Tchakounté, A.D. Wandala, Y. Tiguiane, Detection of android malware based on sequence alignment of permissions, *Int. J. Comput. (IJC)* 35 (1) (2019) 26–36.
- [24] Z. Yuan, Y. Lu, Y. Xue, Droiddetector: android malware characterization and detection using deep learning, *Tsinghua Sci. Technol.* 21 (1) (2016) 114–123.
- [25] Z. Yuan, Y. Lu, Z. Wang, Y. Xue, Droid-sec: deep learning in android malware detection, in: *Proceedings of the 2014 ACM Conference on SIGCOMM*, 2014, pp. 371–372.
- [26] Number of smartphone users worldwide from 2016 to 2021, 2020, <https://www.statista.com/statistics/330695/number-of-smartphone-users-worldwide/> (Accessed: 2020-04-3).
- [27] M.K.A. Abuthawabeh, K.W. Mahmoud, Android malware detection and categorization based on conversation-level network traffic features, in: *2019 International Arab Conference on Information Technology (ACIT)*, IEEE, 2019, pp. 42–47.
- [28] F. Noorbehbahani, F. Rasouli, M. Saberi, Analysis of machine learning techniques for ransomware detection, in: *2019 16th International ISC (Iranian Society of Cryptology) Conference on Information Security and Cryptology (ISCISC)*, IEEE, 2019, pp. 128–133.
- [29] Android malware dataset (cicandmal2017 - first part), 2020, <https://www.unb.ca/cic/datasets/andmal2017.html> (Accessed: 2020-03-12).

- [30] R. Chen, Y. Li, W. Fang, Android malware identification based on traffic analysis, in: *International Conference on Artificial Intelligence and Security*, Springer, 2019, pp. 293–303.
- [31] Intrusion detection evaluation dataset (cicids2017), 2020, <https://www.unb.ca/cic/datasets/ids-2017.html> (Accessed: 2020-03-12).
- [32] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: *ICISSP*, 2018, pp. 108–116.
- [33] M. Samara, E.-S.M. El-Alfy, Benchmarking open-source android malware detection tools, in: *2019 2nd IEEE Middle East and North Africa COMMUNICATIONS Conference (MENACOMM)*, IEEE, 2019, pp. 1–6.
- [34] D. Maiorca, F. Mercaldo, G. Giacinto, C.A. Visaggio, F. Martinelli, R-PackDroid: API package-based characterization and detection of mobile ransomware, in: *Proceedings of the Symposium on Applied Computing*, 2017, pp. 1718–1723.
- [35] P.P. Chan, W.-K. Song, Static detection of android malware by using permissions and api calls, in: *2014 International Conference on Machine Learning and Cybernetics*, Vol. 1, IEEE, 2014, pp. 82–87.
- [36] D.-J. Wu, C.-H. Mao, T.-E. Wei, H.-M. Lee, K.-P. Wu, Droidmat: Android malware detection through manifest and api calls tracing, in: *2012 Seventh Asia Joint Conference on Information Security*, IEEE, 2012, pp. 62–69.
- [37] M. Zhang, Y. Duan, H. Yin, Z. Zhao, Semantics-aware android malware classification using weighted contextual api dependency graphs, in: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, 2014, pp. 1105–1116.
- [38] Y. Zhou, X. Jiang, Dissecting android malware: Characterization and evolution, in: *2012 IEEE Symposium on Security and Privacy*, IEEE, 2012, pp. 95–109.
- [39] A.H. Lashkari, A.F.A. Kadir, H. Gonzalez, K.F. Mbah, A.A. Ghorbani, Towards a network-based framework for android malware detection and characterization, in: *2017 15th Annual Conference on Privacy, Security and Trust (PST)*, IEEE, 2017, pp. 233–23309.
- [40] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and vpn traffic using time-related, in: *Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP)*, 2016, pp. 407–414.
- [41] A. Feizollah, N.B. Anuar, R. Salleh, G. Suarez-Tangil, S. Furnell, Androdialysis: Analysis of android intent effectiveness in malware detection, *Comput. Secur.* 65 (2017) 121–134.
- [42] W.Z. Zarni Aung, Permission-based android malware detection, *Int. J. Sci. Technol. Res.* 2 (3) (2013) 228–234.
- [43] C.-Y. Huang, Y.-T. Tsai, C.-H. Hsu, Performance evaluation on permission-based detection for android malware, in: *Advances in Intelligent Systems and Applications-Volume 2*, Springer, 2013, pp. 111–120.
- [44] G. Canfora, E. Medvet, F. Mercaldo, C.A. Visaggio, Detecting android malware using sequences of system calls, in: *Proceedings of the 3rd International Workshop on Software Development Lifecycle for Mobile*, 2015, pp. 13–20.
- [45] M.K. Alzaylaee, S.Y. Yerima, S. Sezer, Emulator vs real phone: Android malware detection using machine learning, in: *Proceedings of the 3rd ACM on International Workshop on Security and Privacy Analytics*, 2017, pp. 65–72.
- [46] Investigation of the android malware (cicinesandmal2019), 2020, <https://www.unb.ca/cic/datasets/invesandmal2019.html> (Accessed: 2020-03-12).
- [47] M.T. Ahvanooy, Q. Li, M. Rabbani, A.R. Rajput, A survey on smartphones security: Software vulnerabilities, malware, and attacks, 2020, arXiv preprint [arXiv:2001.09406](https://arxiv.org/abs/2001.09406).
- [48] E. Erturk, A case study in open source software security and privacy: Android adware, in: *World Congress on Internet Security (WorldCIS-2012)*, IEEE, 2012, pp. 189–191.
- [49] Android malware genome project, 2020, <http://www.malgenomeproject.org/> (Accessed: 2020-01-12).
- [50] W. Zhou, Y. Zhou, X. Jiang, P. Ning, Detecting repackaged smartphone applications in third-party android marketplaces, in: *Proceedings of the Second ACM Conference on Data and Application Security and Privacy*, 2012, pp. 317–326.
- [51] Q. Liao, Ransomware: a growing threat to smes, in: *Conference Southwest Decision Science Institutes*, 2008.
- [52] T. Micro, Ransomware Definition—Security Intelligence, TREND Micro, Irving, Tex, USA, 2015.
- [53] P. Zavarsky, D. Lindsog, et al., Experimental analysis of ransomware on windows and android platforms: Evolution and characterization, *Procedia Comput. Sci.* 94 (2016) 465–472.
- [54] S. Gupta, Types of malware and its analysis, *Int. J. Sci. Eng. Res.* 4 (1) (2013).
- [55] S. Omeleze, H.S. Venter, Testing the harmonised digital forensic investigation process model-using an android mobile phone, in: *2013 Information Security for South Africa*, IEEE, 2013, pp. 1–8.
- [56] K. Hamandi, A. Chehab, I.H. Elhaji, A. Kayssi, Android sms malware: Vulnerability and mitigation, in: *2013 27th International Conference on Advanced Information Networking and Applications Workshops*, IEEE, 2013, pp. 1004–1009.
- [57] SMS attacks and mobile malware threats, 2020, <https://www.kaspersky.com/resource-center/threats/sms-attacks> (Accessed: 2020-03-12).
- [58] A.H. Lashkari, A.F.A. Kadir, L. Taheri, A.A. Ghorbani, Toward developing a systematic approach to generate benchmark android malware datasets and classification, in: *2018 International Carnahan Conference on Security Technology (ICCST)*, IEEE, 2018, pp. 1–7.
- [59] M. Usman Sarwar, A. Rehman Javed, F. Kulsoom, S. Khan, U. Tariq, A. Kashif Bashir, Parciv: Recognizing physical activities having complex interclass variations using semantic data of smartphone, *Softw. - Pract. Exp.* (2020).



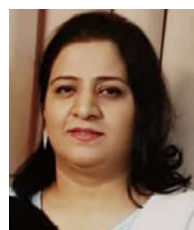
Syed Ibrahim Imtiaz is a MS scholar at National Center for CyberSecurity, Air University, Islamabad, Pakistan. He is currently pursuing his degree in Masters from Air University, Islamabad, Pakistan. His current research interests include but are not limited to cybersecurity, artificial intelligence, computer vision, network security, IoT, smart city, and application development for smart living. He aims to contribute to interdisciplinary research of computer science and human-related disciplines.



Saif ur Rehman is a student at the Faculty of Computing and AI, Air University, Islamabad, Pakistan. He is currently pursuing his degree in Bachelor of Science in Computer Science from Air University, Islamabad, Pakistan. His current research interests include but are not limited to cybersecurity, artificial intelligence, computer vision, network security, IoT, smart city, and application development for smart living. He aims to contribute to interdisciplinary research of computer science and human-related disciplines.



Abdul Rehman Javed is a lecturer at the Department of CyberSecurity, Air University, Islamabad, Pakistan. He worked with National Cybercrimes and Forensics Laboratory, Air University, Islamabad, Pakistan. He received his Master's degree in Computer Science from National University of Computer and Emerging Sciences, Islamabad, Pakistan. He is a reviewer of many well-known journals, including, Sustainable cities and society (Elsevier), Journal of Information Security and Applications (Elsevier) and IEEE Access. His current research interests include but are not limited to mobile and ubiquitous computing, data analysis, knowledge discovery, data mining, natural language processing, smart homes, and their applications in human activity analysis, human motion analysis and e-health. He aims to contribute to interdisciplinary research of computer science and human-related disciplines. He has authored more than over 10 peer-reviewed articles on topics related to cybersecurity, mobile computing and digital forensics.



Dr. Zunera Jalil received the B.Sc. degree from Punjab University, Lahore, Pakistan, in 1999, and then Master's degree in computer science from International Islamic University, Islamabad, Pakistan. She later earned scholarship from Higher Education Commission of Pakistan to pursue M.S. degree in computer science and then Ph.D. degree in computer science with information security specialization from the FAST-National University of Computer and Emerging Sciences, Islamabad, Pakistan, in 2007 and 2010, respectively. She served at International Islamic University, Islamabad, Iqra University, Islamabad and then Saudi Electronic University, Riyadh, Saudi Arabia. She is currently with the Department of CyberSecurity and is involved with National Cybercrimes and Forensics Laboratory, Air University, Islamabad, Pakistan. Her current research interests include but are not limited to computer forensics, intelligent systems, and data privacy protection.



Xuan Liu graduated from Shandong University, China, and received M.S. degree from Wuhan Polytechnic University, China and Ph.D. degree in computer science and engineering from Southeast University, China. Since 2020, he joins Yangzhou University, China. He is serving as an Advisory Editor of Wiley Engineering Reports, an Associate Editor of Springer Telecommunication Systems, IET Smart Cities, Taylor & Francis International Journal of Computers and Applications and KeAi International Journal of Intelligent Networks, an Area Editor of EAI Endorsed Transactions on Internet

of Things, the Lead Guest Editor of Elsevier Internet of Things, Wiley Transactions on Emerging Telecommunications Technologies and Wiley Internet Technology Letters, and the Chair of CollaborateCom 2020 workshop. He serves/served as a TPC Member of ACM MobiCom 2020 workshop, IEEE INFOCOM 2020 workshop, IEEE ICC 2020/2019, IEEE Globecom 2019, IEEE PIMRC 2020/2019, IEEE MSN 2020, IEEE VTC 2020/2019/2018, IEEE ICIN2020, IEEE GIIS 2020, IEEE DASC 2019, APNOMS 2020/2019, Ad Hoc-Now2020, FNC 2020/2019, EAI CollaborateCom 2020/2019, and EAI ChinaCom 2019, etc. Furthermore, he has been reviewing papers for 20+ reputable conferences/journals including IEEE INFOCOM, IEEE ICC, IEEE Globecom, IEEE WCNC, IEEE PIMRC, IEEE COMMAG, IEEE TI, IEEE

IoT, IEEE CL, Elsevier, JNCA, Elsevier FGCS, Springer WINE, Springer TELS, IET SMC, EAI CollaborateCom, and Wiley IJCS, etc. His research interests include content network and governance, collaborate networking for smart cities, aerial communication systems, etc.



Waleed S. Alnumay received his bachelor degree in Computer Science from King Saud University, Riyadh, Saudi Arabia in the year 1993. He did his master degree in Computer Science from University of Atlanta, Atlanta, Georgia, USA in the year 1996. He completed his Ph.D. in Computer Science from Oklahoma University, Norman, Oklahoma, USA in the year 2004. Dr. Alnumay is currently working as an Associate Professor of Mobile Networking in Computer Science Department, King Saud University. He has published research papers in reputed international conferences

and journals. His main research interest is Computer Networks and Distributed Computing that includes but not limited to Mobile Ad hoc and Sensor Networks, Information-Centric Networking and Software-Defined Networking.